Understand the project. Explain it with confidence.

Viva and code explanation guide

A complete study companion covering the data, code, models, mathematics, results, limitations and demonstration. Read the numerical examples aloud and rehearse the dashboard sequence.

Contents

1. Your opening explanation 3

2. The networking problem 3

3. Dataset and why we changed it 4

4. Quality decisions and exclusions 4

5. Inputs, feature engineering and windows 5

6. Splitting, scaling and leakage 6

7. Baselines 6

8. LSTM explanation and exact implementation 6

9. GRU explanation 7

10. Transformer explanation 7

11. Training: explain each step 8

12. Metrics with a numerical example 9

13. Results you should know 9

14. Allocation formulas and policy selection 10

15. Code walkthrough: what each file does 11

16. Saved files and what they prove 13

17. Verification: defend the evidence accurately 14

18. Demonstration commands and sequence 14

19. Likely viva questions with defensible answers 15

20. What to revise first tonight 17

Reference material already in the project 17

Prepared for the completed implementation. Study this alongside the source and the saved results, rather than memorizing claims without understanding them.

1. Your opening explanation

"Our project forecasts future recorded Internet traffic using historical measurements. We compare LSTM, GRU and a small Transformer against persistence and moving-average baselines. We then use ten-minute forecasts in an offline capacity-allocation simulation. We evaluate both prediction errors and the tradeoff between insufficient and unused capacity. A Streamlit dashboard presents saved results and historical replay."

Add: "The actual experiment uses one institution from CESNET-TimeSeries24. Cloud, ISP and data-center use are potential applications; we have not validated deployments in all three."

The pipeline is:

```
Raw byte counts + timestamp mapping
  -> audit and timestamp blocks
  -> outage/boundary eligibility flags
  -> demand/calendar/rolling features
  -> chronological splits and training-only scaling
  -> three-hour input windows
  -> baselines and separately trained direct neural forecasts
  -> original-unit forecasting metrics
  -> ten-minute equal-margin allocation simulation
  -> saved artifacts -> dashboard, report and slides
```

2. The networking problem

Bandwidth capacity is the rate a resource can support. Observed throughput is the traffic rate actually recorded. Demand can exceed observed throughput if a network is already constrained, so our recorded rate is a practical demand proxy, not a direct measurement of unconstrained offered demand.

A reactive policy provisions from the most recent measurement. A proactive policy provisions from a forecast. Both can miss an unexpected spike. Forecasting alone does not reserve capacity: an operational system would need an actual controller and resource APIs, which are outside this implementation.

Congestion, latency, packet loss and QoS motivate the project, but we do not claim to have measured improvements in those quantities. We measure forecast error and simulated capacity shortfall.

Distinguish:

Term	Meaning in this project
Traffic volume	Recorded bytes over a nominal interval
Traffic rate	Converted nominal Mbps
Forecast	A model's estimate of a future interval's rate
Allocated capacity	Simulated Mbps chosen from that forecast and margin
Shortfall	Positive difference between actual recorded rate and capacity
Physical link utilization	Rate divided by a known link capacity; not available here
Historical replay	Showing held-out recorded intervals, not live network monitoring

3. Dataset and why we changed it

Final dataset: CESNET-TimeSeries24 v1, Zenodo record 13382427. Attribution and CC BY 4.0 information are in DATA_ATTRIBUTION.md. Its institution 0 ten-minute series is used. GÉANT 2005 was an earlier proposal; its assumptions and results are not part of the final experiment.

Two source members are extracted: institutions/agg_10_minutes/0.csv and times/times_10_minutes.csv. Locally they are institution_0.csv and times_10_minutes.csv. Join by id_time, which is an identifier requiring the separate mapping, not an assumed Unix timestamp.

Facts to remember:

- 40,298 raw observations; 19 source columns.
- Supplied timestamps span 9 October 2023 to 14 July 2024.
- Nominal interval: 600 seconds = 10 minutes.
- All 283 institutions were compared for coverage and prospective window availability before training; 56 had complete supplied time-ID coverage, 229 had at least 95%.
- Institution 0 was chosen on availability and completed inspection, not because a model happened to score well on it.
- Its sample has no missing or zero byte measurements. That does not prove complete collection or absence of anomalies.
- Five irregular UTC transitions create six regular blocks. We do not claim to know the cause of every discontinuity.

The rate conversion is:

```
Mbps = n_bytes * 8 / (600 * 1,000,000)
```

Example: 7,500,000,000 bytes in ten minutes gives 100 Mbps. Bytes become bits by multiplying by eight; divide by seconds to get bits/second; divide by one million for decimal Mbps.

The source includes both directions. Do not call the target exclusively download bandwidth. Do not sum institutions and claim unique network-wide demand: endpoint accounting may double-count traffic, and this was not resolved. Do not substitute a timestamp gap into the denominator, because a gap does not establish how long bytes were captured.

4. Quality decisions and exclusions

The dataset paper reports a monitoring failure around 21 May to 4 June 2024. Exact affected-institution and timezone scope are uncertain. Our explicit conservative policy excludes all supplied-calendar dates 20 May to 5 June inclusive: 17 days, 2,448 observations.

We also guard the first and last record of each of six blocks: 12 additional, disjoint observations. Thus 40,298 - 2,448 - 12 = 37,838 eligible individual observations.

These are policy exclusions, not an automated outlier detector. Raw values are retained. We do not delete every large traffic spike merely because it is unusual: legitimate spikes are part of the prediction problem.

When constructing a window, the entire path from the first history record to the future target must stay inside one regular block and avoid exclusions. Checking only the input endpoints and target would allow a hidden gap in between.

Input missingness is handled by causal forward filling within allowed history. At a boundary or exclusion the carried value resets. Future values are never used to fill the past. Missing target observations are excluded rather than

filled. In the actual institution 0 byte series no imputation is needed, but the logic is implemented and tested.

5. Inputs, feature engineering and windows

Every neural input is 18 records by eight features: three hours at six records per hour.

Feature	Purpose
input_traffic_mbps	Recent traffic magnitude
hour_sin, hour_cos	Daily clock position represented cyclically
weekday_sin, weekday_cos	Weekly calendar position represented cyclically
input_missing	Distinguish a carried input from an observed one
rolling_mean_6	Recent local traffic level
rolling_std_6	Recent variability, population standard deviation

For a clock fraction q, cyclical encoding uses sin(2*pi*q) and cos(2*pi*q). Midnight and late evening are nearby on this circle, whereas raw hours 0 and 23 appear far apart. Hour fraction includes supplied hours, minutes and seconds. Calendar features use the supplied offset; alignment and charts use UTC.

The rolling statistics use at most six observations through the current historical position, inside each input window. At the first position there is only one value: mean equals that value and standard deviation is zero. They do not look beyond their input position. Target future calendar features are not separately added.

The missing flag is constant zero in this experiment. Standardizing a constant feature uses standard deviation one, preventing division by zero. Keeping the flag supports the pipeline's general missing-input handling; it does not contribute varying information in this sample.

Target meaning: the most important alignment detail

Models predict one future ten-minute interval's nominal average rate. A 60-minute horizon does not mean the average or total over the next hour.

Example: at forecast origin 13:00, the latest available interval ended at 13:00. The three-hour input covers 10:00-13:00:

Horizon	Predicted interval
10 minutes	13:00-13:10
30 minutes	13:20-13:30
60 minutes	13:50-14:00

An interval is assumed available at its supplied start plus 600 seconds. This is an offline assumption; real collector/export delays are not verified.

Using array indices: origin = target - lead, start = origin - 17. Lead is 1, 3 or 6 records. X is the 18 records from start through origin; y is the single target rate. Training makes nine neural models: three architectures times three horizons. These are direct forecasts, not recursive predictions fed back as future inputs.

6. Splitting, scaling and leakage

Chronological timeline cuts are 70% training, 15% validation and 15% test before exclusions. A window's target completion determines its split. Exclusions mean usable-window proportions need not remain exactly 70/15/15.

Lead	Training	Validation	Test
10 min	28,129	4,029	5,554
30 min	28,121	4,029	5,548
60 min	28,109	4,029	5,539

Validation starts 21 April 2024 22:40:52 UTC. Nominal test starts 2 June 2024 22:10:52 UTC, but the outage policy removes its initial portion. Later valid targets also require a complete unexcluded history-to-target path.

Do not randomly split a time series: that can allow future patterns and strongly overlapping histories to inform predictions nominally evaluated in the past. We shuffle only already-selected training windows for optimization; no validation/test windows join training.

Validation selects the best epoch and allocation margin. Test is used for final reporting. It must not determine model tuning or retrospective choice of a favorable margin.

Feature standardization: z = (x - training_mean) / training_std. Target has its own mean and standard deviation. Each horizon fits its own scalers on training windows only, with feature statistics over their historical positions. Overlapping observations can appear more than once because the learner also receives overlapping windows. Inverse prediction: predicted_Mbps = predicted_scaled * target_std + target_mean.

Earlier observed training history may appear in a validation input; earlier validation history may appear in a test input. That is permitted when the observations were already available at the forecast origin. Training weights and scalers still use training targets/windows only.

Leakage safeguards: no backward filling; no future rolling values; training-only scaler fitting; chronological target cuts; no gap-crossing windows; validation-only checkpoint/margin decisions. Same timestamped targets and actual rates are enforced across all five predictors within a horizon.

7. Baselines

Persistence: prediction = latest observed input rate. Moving average: prediction = mean of the last four input rates, corresponding to 40 nominal minutes. Both apply to every horizon with the same origin and target as neural models. Neither has trained parameters or a fitted optimization phase.

Baselines answer whether model complexity is justified. Persistence can be strong on slowly changing demand. Moving average suppresses noise but can lag transitions and smooth spikes. A neural model is not automatically superior.

8. LSTM explanation and exact implementation

A basic recurrent network updates a hidden state for every observation. Backpropagation through many recurrent steps can make gradients shrink or grow. LSTM introduces a separate cell state and gates to control information flow; it helps learn temporal dependencies but does not guarantee perfect long-range memory or forecasting.

At time t, concatenate current features x_t and previous hidden state h_prev conceptually. A sigmoid gate outputs values between zero and one; tanh supplies bounded candidate content:

```
forget gate f = sigmoid(W_f [x_t,h_prev] + b_f)
input gate  i = sigmoid(W_i [x_t,h_prev] + b_i)
candidate   g = tanh(W_g [x_t,h_prev] + b_g)
output gate o = sigmoid(W_o [x_t,h_prev] + b_o)
cell c_t = f * c_prev + i * g
hidden h_t = o * tanh(c_t)
```

Multiplication is elementwise. Forget gates regulate retained memory, input gates regulate written content, and output gates regulate exposed hidden state. These equations explain the model family; PyTorch combines gate matrix calculations internally.

Our code uses nn.LSTM(input_size=8, hidden_size=64, num_layers=2, batch_first=True, dropout=0). Layer two receives layer one's output sequence. The final historical position from layer two goes through Linear(64,1). The hidden and cell states start fresh for every window because no previous state is passed to the layer. It is unidirectional, not bidirectional or stateful across batches.

Shapes for a full batch: X [64,18,8] -> recurrent output [64,18,64] -> last position [64,64] -> linear [64,1] -> squeeze [64]. The final batch can contain fewer than 64 windows. In a two-layer LSTM, final h and c each have shape [2,batch,64], though the code reads the output sequence instead.

Parameter count 52,289: first recurrent layer 18,944; second 33,280; scalar head 65. PyTorch stores separate input and recurrent bias vectors, which must both be counted.

9. GRU explanation

GRU also controls recurrent information with gates but has no separate LSTM-style cell state. It uses a reset gate, update gate and candidate hidden state. In the PyTorch convention:

```
r = sigmoid(W_ir*x + b_ir + W_hr*h_prev + b_hr)
z = sigmoid(W_iz*x + b_iz + W_hz*h_prev + b_hz)
n = tanh(W_in*x + b_in + r*(W_hn*h_prev + b_hn))
h = (1-z)*n + z*h_prev
```

A large z preserves more previous state in this convention. Textbooks sometimes define the update gate oppositely, so explain the convention instead of mixing formulas.

Our nn.GRU has the same input size, two layers, 64 hidden units, dropout zero and scalar final-position head as LSTM. It resets state per window. There are 39,233 trainable parameters: 14,208 + 24,960 + 65. Fewer parameters do not guarantee lower measured runtime or lower error; hardware, implementation and stopping epochs matter.

10. Transformer explanation

Self-attention lets historical positions weight information from other historical positions rather than processing the history only by recurrent updates. Linear projections form queries Q, keys K and values V. A head computes:

```
Attention(Q,K,V) = softmax(Q*K_transpose / sqrt(d_k)) * V
```

Queries and keys determine relevance weights; values carry the information being combined. Multiple heads can learn different relationships. Our model has dimension 64 and four heads, so each head dimension is 16.

Implementation: Linear(8,64) projection, fixed sinusoidal position encodings for 18 historical positions, two separately constructed TransformerEncoderLayer instances, and Linear(64,1) on the last position. Each encoder layer uses attention, residual connections, post-layer normalization and a 128-unit feed-forward block with GELU. Dropout is zero. Total trainable parameters: 67,585. Positional encodings are saved as a nontrainable buffer.

Attention alone does not encode chronological position; the sine/cosine position encoding distinguishes positions. It differs from calendar features: position encoding describes location inside the input sequence, while calendar

features describe clock/weekday.

There is no future target token. Full attention inside this historical window is safe because every supplied observation is available at forecast origin. Therefore a triangular causal mask is not needed for this sequence-to-one historical encoder. This would change if training included future tokens or per-position forecasts with different availability.

Shapes: [batch,18,8] -> projection [batch,18,64] -> two encoder layers [batch,18,64] -> last position [batch,64] -> scalar forecast [batch]. This is a small encoder adaptation, not the original encoder-decoder translation Transformer and not an LLM.

The code requests the mathematical SDPA attention backend and disables MHA inference fastpath for the experiment to keep attention implementations consistent. No claim is made that this is the fastest attention configuration.

11. Training: explain each step

Settings: seed 42, Adam learning rate 0.001, batch size 64, scaled-target MSE, maximum 30 epochs, patience five, minimum improvement zero, dropout zero. Model capacity and settings were fixed across horizons; no extensive tuning study was performed.

1. Load prepared arrays, metadata, scalers and preparation configuration.
2. Check dimensions, finite values, split consistency and matching comparison-file provenance.
3. Seed Python, NumPy and PyTorch; configure deterministic operations.
4. Build model, Adam optimizer and a seeded shuffled training DataLoader.
5. For a batch, run forward prediction and calculate mean squared error against scaled targets.
6. Clear accumulated gradients using zero_grad, backpropagate with loss.backward, update weights with optimizer.step.
7. After the epoch, predict validation windows with evaluation mode and no gradients.
8. If validation MSE improves, save the state_dict checkpoint and reset patience; otherwise increment the counter.
9. Stop after five consecutive non-improving epochs, or the epoch limit.
10. Reload the lowest-validation-MSE checkpoint, inverse-scale forecasts, calculate original-unit metrics and save evidence.

Epoch means one pass over the training windows; batch means a subset used for one gradient update; learning rate controls update scale; optimizer uses gradients to change parameters; loss is the objective optimized by training. Adam maintains estimates of gradient first and second moments to adapt updates.

Backpropagation through time differentiates recurrent computation across the 18-step window. Prediction uses torch.inference_mode, so gradients are not built. model.eval switches relevant layers to evaluation behavior; it does not itself disable gradients. Both are used appropriately.

Training MSE is measured while weights evolve across batches; validation MSE uses the resulting fixed epoch model on a different distribution. Validation loss can be lower than training loss, and that alone does not prove leakage.

No dropout was used. Early stopping limits training duration based on validation, but it does not eliminate all overfitting. A fixed seed improves repeatability; it does not estimate uncertainty or prove a model advantage is statistically significant.

Horizon	Model	Epochs completed	Restored best epoch
10	LSTM	20	15
10	GRU	18	13
10	Transformer	13	8
30	LSTM	6	1
30	GRU	9	4
30	Transformer	9	4
60	LSTM	12	7
60	GRU	12	7
60	Transformer	12	7

12. Metrics with a numerical example

For actual y, prediction p and n evaluated targets:

```
MAE  = sum(abs(p-y)) / n
RMSE = sqrt(sum((p-y)^2) / n)
MAPE = 100 * mean(abs(p-y)/y), for y>0 only
R2   = 1 - sum((p-y)^2) / sum((y-mean(y))^2)
```

MAE and RMSE have Mbps units. RMSE weights large errors more heavily. MAPE is a percentage and can emphasize low-demand observations. R2 is dimensionless, can be negative, and is not classification accuracy. R2=0.72 does not mean 72% of packets or forecasts were correct.

Example: actual [100,200], predictions [110,180]. Absolute errors [10,20] give MAE 15 Mbps, RMSE sqrt(250)=15.811 Mbps, MAPE 10%, and R2=1-500/5000=0.9. These are illustrative values, not experiment results.

Targets of zero make ordinary percentage error undefined, so the implementation uses only positive targets and reports included/excluded counts. All actual evaluated targets here are positive. R2 is undefined for a constant target or fewer than two observations; undefined values remain blank rather than being invented.

Forecasts are not clipped before accuracy evaluation. Allocation separately clamps negative forecasts to zero. No negative predictions occurred in the saved runs, but the policy still defines that case.

13. Results you should know

Test RMSE in Mbps:

Predictor	10 min	30 min	60 min
Persistence	38.629	41.081	43.754
Moving average	32.478	35.060	38.668
LSTM	31.175	33.611	35.091

Predictor	10 min	30 min	60 min
GRU	31.006	32.290	34.785
Transformer	30.803	32.340	38.537

Transformer wins test MAE/RMSE at ten minutes. GRU wins them at thirty and sixty minutes. Moving average wins ten-minute MAPE. LSTM wins sixty-minute MAPE. Thirty-minute GRU also has the lowest test MAPE. These statements refer to this particular fixed-seed experiment.

At sixty minutes, Transformer has MAE 29.235 Mbps and MAPE 32.53%, both worse than persistence, despite RMSE 38.537 Mbps being below persistence's 43.754. Different error distributions can produce different rankings.

Neural validation RMSE rankings differ from test rankings: GRU lowest at ten minutes, Transformer at thirty, GRU at sixty. Test rankings must not retroactively select checkpoints or change settings.

MAE and RMSE increase with lead in the saved runs, but horizon target sets differ slightly. Do not claim a perfectly paired causal comparison or that every metric must worsen monotonically. Do not claim the Transformer degraded for a proven specific reason: limited data, inductive bias, distribution change or optimization are possible explanations that need additional experiments.

The title foregrounds LSTM because it is the original primary model. A fair evaluation can find GRU or a baseline better. The research objective is a defensible comparison, not making LSTM win.

CPU inference uses 25 warm-up and 1,000 timed single-window calls with preloaded inputs, one CPU thread for neural models, and includes inverse scaling. Loading, feature preparation, dashboard rendering and collector delay are excluded. Training duration covers epochs, validation and improved-checkpoint writes. Compare timing cautiously: completed epoch counts and machine load differ.

14. Allocation formulas and policy selection

For predicted demand p, actual y and safety margin m:

```
capacity c = max(0,p)*(1+m)
deficit d = max(y-c,0)
unused u = max(c-y,0)
served s = min(y,c)
```

Reactive p is the most recent observed input (persistence). Four proactive alternatives use moving average, LSTM, GRU or Transformer. All policies are evaluated at the same margins 5%, 10%, 15%, 20%, separately on validation and test: 5 policies * 4 margins * 2 splits = 40 metric rows.

For each policy, among validation candidates with shortfall frequency <=1%, select the lowest mean allocated capacity. If none qualifies, use 20% and explicitly flag the target as unmet. Test data never chooses the margin.

All five policies fail this validation criterion at all tested margins. Thus 20% is a fallback, not a guaranteed reliability level and not a globally optimized allocation.

Reported quantities:

- Shortfall frequency: fraction of intervals with d>0, expressed as percent; measures how often capacity is insufficient, not how much.
- Deficit volume: sum(d)*600/8 decimal MB; measures the simulated amount of unserved demand over evaluated intervals.
- Mean capacity: mean(c) Mbps.

- Over-provisioning: mean(u) Mbps and sum(u)*600/8 MB.
- Served-demand-to-capacity ratio: sum(s)/sum(c), undefined for zero total capacity. It is a weighted aggregate ratio, not mean of per-interval ratios.

Why divide by eight? Mbps times seconds gives megabits; eight bits make one byte. Our intervals are 600 seconds. Do not use 900 seconds from the abandoned GÉANT plan.

Hand calculation: forecast 100 Mbps, margin 10%, actual 200 Mbps. Capacity=110 Mbps, shortfall=90 Mbps, served=110 Mbps, unused=0, deficit=90*600/8=6,750 MB. With actual 100 instead, unused=10 Mbps, deficit zero and served=100 Mbps.

Actual test allocation at the common 20% fallback

Policy	Shortfall intervals	Shortfall %	Mean capacity Mbps	Mean unused Mbps
Reactive	1,302	23.443	140.214	28.478
Moving average	1,047	18.851	140.191	27.394
LSTM	603	10.857	148.669	34.472
GRU	616	11.091	147.985	33.886
Transformer	609	10.965	146.690	32.598

All policies use 5,554 identical test intervals. LSTM has the fewest shortfalls but the largest mean capacity in this comparison. Reactive-to-LSTM shortfall reduction is about 12.59 percentage points, not 12.59% relative. Mean capacity rises about 8.46 Mbps. These numbers do not establish a cost-free efficiency improvement.

Transformer wins ten-minute forecast RMSE but LSTM wins twenty-percent test shortfall frequency. Symmetric average squared error and a one-sided capacity threshold optimize different things; upward forecast bias can reduce shortfalls while allocating more unused capacity. Investigating causal explanations needs additional analysis.

Deficit volume is not measured packet loss: this simulation has no queues, packet retransmission, physical routes or finite provisionable capacity limit. Allocation is evaluated only at ten minutes. Thirty/sixty-minute forecast availability does not imply those allocation experiments exist.

15. Code walkthrough: what each file does

All paths below are relative to the project root.

File	What to explain
data_processing/download_data.py	Fetch source archives, check MD5, safely extract exact named sample members, preserve differing existing files
data_processing/audit_sample.py	Inspect IDs, values, timestamps and intervals before transformations
data_processing/mark_segments.py	Join mapping, convert UTC, label uninterrupted blocks in an inspection copy
data_processing/inspect_outage.py	Compare measurements around the reported incident
data_processing/mark_exclusions.py	Add the documented date flags while retaining raw observations

File	What to explain
data_processing/compare_institutions.py	Pre-model selection by availability; full archive needed to repeat all-institution audit
data_processing/build_features.py	Convert bytes, enforce flags, causal input carry, clock features, boundary guards
data_processing/prepare_dataset.py	Build windows and splits, fit scalers, save arrays/configuration/checks/quality plot
models/lstm_model.py	Two-layer LSTM and final-position linear readout
models/gru_model.py	Matching GRU architecture and readout
models/transformer_model.py	Projection, position buffer, two attention layers and scalar readout
experiments/forecast_metrics.py	Original-unit metrics and explicit undefined/positive-target handling
experiments/evaluate_baselines.py	Persistence/moving-average forecasts, timing and comparison plot
experiments/train_neural.py	Shared training loop, device, seeded batches, validation stopping, checkpoints, outputs
experiments/train_lstm.py, train_gru.py, train_transformer.py	Small model-specific command entry points into shared training
experiments/evaluate_allocation.py	Forecast joins, provenance, equal-margin replay, validation defaults and policy scores
dashboard/artifacts.py	Read/validate saved CSV/JSON, align target identities, recompute scores, detect stale allocation sources
dashboard/charts.py	Actual UTC charts with gaps broken instead of drawn across
dashboard/app.py	Streamlit sidebar controls, replay, comparison, allocation and downloads
dashboard/styles.css, .streamlit/config.toml	Appearance and local server settings
scripts/reproduce.py	Whole experiment in dependency order; fresh mode archives outputs before regeneration
scripts/verify_results.py	Reconstruct prepared evidence, check predictions/metrics/checkpoints and allocation replay, bind hashes
scripts/summarize_results.py	Combined 30-row forecasting table and cross-horizon scientific plot
scripts/package_submission.py	Package verified deliverables with hashes, rejecting changed evidence
scripts/record_demo.cjs	Optional Playwright/Edge recording helper; not a training dependency
tests/	41 checks covering leakage, metrics, models, allocation, dashboard and corrupted evidence
docs/build_report.py	Generate manuscript/tables from saved experiment artifacts
docs/presentation/	Slide generator, speaker notes and rehearsal script

Important prepare_dataset functions: select_windows applies eligibility to the complete path; window_features builds causal per-window rolling statistics; fit_scalers uses split==train only; main orchestrates outputs. Important

training functions: predict batches inference without gradients; validation_mse scores scaled targets; main handles learning/checkpoint selection. Important allocation functions: align_forecasts validates IDs/times/actual rates; allocate performs arithmetic; select_margin uses validation results only.

ROOT = Path(__file__).resolve().parents[1] finds the project root relative to source location, allowing the project to move. The if __name__ == '__main__' block runs a file's command entry point only when executed as a script/module. python -m executes using the package context, keeping imports consistent.

16. Saved files and what they prove

Artifact	Meaning
Raw CSVs	Original sample values and mapping
institution_0_features.csv	Derived rates/features and eligibility flags
windows_<horizon>min.npz	Compressed X, scaled y, original target/input rates, split codes, start/origin/target indices
windows_<horizon>min.csv	Human-readable IDs, timestamps and split assignment
scalers_<horizon>min.json	Train-fitted means and standard deviations
preparation_config.json	Units, horizons, split/exclusion policy and raw SHA256 hashes
best_model.pt	Trained parameter state_dict, not the source architecture or a raw dataset
training_history.csv	Training/validation loss per epoch and best-checkpoint decisions
predictions.csv	Forecast origin, target identity/times, actual/predicted Mbps and split
metrics.csv	Full-split forecasting or allocation scores
run_config.json	Settings, versions, training/timing metadata and provenance
comparison.csv	Matched metrics from model families, not new independent experiments
allocation replay.csv	Per-policy/margin/interval capacity, served rate and deficits
default_test_metrics.csv	Test scores at each policy's validation-selected margin
final_verification.json	Verification outcome and evidence hashes
reproduction_acceptance.json	Clean-workflow comparison with original outputs

A hash identifies content and detects changes; it does not prove the scientific measurement is correct. A checkpoint requires matching architecture, features and scalers to be useful.

Saved results allow the dashboard to start without model training, raw-data processing, CUDA or even importing PyTorch. Streamlit runs the interface; Plotly draws interactive graphs; pandas handles interface tables; NumPy computes arrays and metrics; Matplotlib produces saved figures; PyTorch trains the three neural families. TensorFlow is not used. CUDA is a GPU computation platform; PyTorch is the model framework that can use it. A virtual environment isolates Python packages; Git tracks source revisions, not the active Python environment.

17. Verification: defend the evidence accurately

41 tests passed, including causal preprocessing, scaler leakage, window-path exclusions, metric edge cases, checkpoint restoration, finite gradients, recurrent state isolation, Transformer positions/layer initialization, flat/spike allocation arithmetic, defaults, dashboard controls and corrupted artifacts.

A separate clean project copy reused the existing raw sample and installed environment, regenerated all fifteen predictor/horizon combinations, and matched all nine neural checkpoint tensor sets and forecast values. All forty allocation score rows and 191,660 replay rows also matched. Fifteen here means five predictors times three horizons, not fifteen neural models. Thirty forecasting metric rows means five predictors times three horizons times two evaluated splits. Replay rows mean five policies times four margins times (4,029+5,554) intervals.

This was not a new internet download or clean dependency installation. Timing values vary with system load and were excluded from accuracy equality checks.

Strict saved-CUDA-checkpoint replay passes on CUDA. CPU replay can differ beyond the strict numeric tolerance; CPU training is supported, and CPU-only viewers can validate saved arithmetic without checkpoint replay. Do not promise exact cross-hardware identity.

18. Demonstration commands and sequence

In Git Bash:

```
cd /d/Projects/Internet-Traffic-Forecasting-for-Intelligent-Bandwidth-Prediction
source VirtEnv/Scripts/activate
python -m streamlit run dashboard/app.py
```

Open http://127.0.0.1:8501. Keep the terminal running. Ctrl+C stops the server.

Demonstrate in this order:

1. Ten-minute test LSTM: explain recorded demand, target, three-hour history and the full-split metric cards.
2. Comparison: show the baselines and Transformer; explain that different metrics have different winners.
3. Forecasts/replay: move the slider and say chart history changes, while scores still cover the complete eligible split.
4. Sixty-minute GRU: show RMSE 34.785 Mbps. Explain the single future interval and longer-lead uncertainty.
5. Return to ten-minute Allocation: choose LSTM and 20%; compare reactive/proactive shortfall and capacity.
6. Change margin to 5%; explain less allocated capacity can produce more shortfalls. Restore 20% and explain the unmet validation-target warning.
7. Choose GRU or Transformer allocation; show the comparison is implemented for all four proactive predictors.
8. Explain limitations and show where generated CSVs/checkpoints/configuration are stored.

Do not start a fresh full retraining immediately before or during the demonstration. The submitted experiment is already saved. Keep the report PDF, slide PDF and deliverables/demo/dashboard_demo.webm as offline backups.

Useful commands if asked:

```
# Unit checks
python -m unittest discover -s tests -v
# Saved artifact checks without neural replay
python -m scripts.verify_results
# Replay saved checkpoints on their training backend
python -m scripts.verify_results --checkpoints --device cuda
# Example training command; changes that model's saved outputs
```

```
python -m experiments.train_lstm --horizon 10 --device cuda
# Complete fresh reproduction, preserving older generated outputs
python -m scripts.reproduce --download --fresh --device auto
```

Do not casually rerun the individual training command in the submitted folder: regenerating outputs can invalidate matching allocation hashes and report provenance until the rest of the workflow is regenerated. Use the full documented workflow or a separate copy for a new experiment.

Troubleshooting: if python cannot import streamlit, confirm VirtEnv is activated and install requirements-dashboard.txt. If no saved results appear, verify the current project folder and results directory or extract the complete ZIP. If port 8501 is occupied, use --server.port 8502 and open that port. CUDA unavailable does not prevent the saved-results dashboard. A stale-artifact error means evidence changed; it is not fixed by deleting validation logic. Use the coherent submission bundle or regenerate consistently.

19. Likely viva questions with defensible answers

What is the research contribution? A reproducible comparison and an explicit connection between forecast performance and capacity tradeoffs, with provenance and measurement safeguards. We do not claim a novel neural architecture or a globally novel allocation algorithm.

Why LSTM? Its gated memory provides a reasonable temporal forecasting baseline for recent demand patterns. We test it against alternatives instead of assuming it is best.

Why GRU and Transformer? They compare a smaller gated recurrent family with an attention-based family under the same input history, horizons and evaluation targets.

Why not a plain RNN? LSTM/GRU were prioritized as gated recurrent models; a plain RNN ablation was not included. We cannot claim measured superiority over an untested model.

Why not ARIMA, seasonal naive or XGBoost? They are useful additional baselines but outside this compact experiment. This limits claims about being the best forecasting method.

Why three hours, 64 units, 0.001 and these margins? They are fixed prototype choices for manageable computation and an understandable comparison. We did not prove they are optimal through tuning.

Why ten rather than fifteen minutes? The final dataset has nominal ten-minute sampling. Forecast and deficit calculations match that interval rather than retaining the earlier GÉANT plan.

Why not use all institutions? One audited series makes the pipeline tractable and explainable. We evaluated candidate coverage, but generalization requires training/testing on more institutions.

Is four months enough? The final series spans October 2023-July 2024, rather than the earlier GÉANT subset. Sufficient data cannot be established from duration alone; coverage, seasonality, drift and model capacity matter. Generalization is limited by the observed period.

Why chronological splits? Forecasts must use information available before the target; temporal holdout better mimics deployment than randomly mixing the timeline.

Why can validation/test use older split history? Earlier observed data is available at forecast time. That does not permit training weights/scalers on held-out targets.

Does training-window overlap invalidate evaluation? Overlap is normal in sliding forecasting windows. Splits follow target time and all feature availability is causal. Overlap does reduce independence, so we do not claim independent-sample significance tests.

Why is the missing flag present when nothing is missing? It supports the explicit missing-input design and remains constant in this particular audited series; it is not evidence that imputation improved results.

Why were those dates removed? The collection paper identifies an incident, and we chose a disclosed buffer because exact scope/timezone was unresolved. A sensitivity study with different policies would strengthen conclusions.

Did you remove outliers to improve scores? No statistical outlier deletion was used. Exclusions follow documented incident dates and timestamp-boundary rules, not favorable model residuals.

Why does Transformer not use a causal mask? Every token is historical relative to the single future target. The model cannot see unprovided target observations.

Why the last-position head? It is a simple fixed readout for sequence-to-one forecasting. In the Transformer, attention lets that position integrate the supplied history; in recurrent models it follows sequential processing.

Why does LSTM not win despite the title? The title describes the original primary approach. Fair evaluation reports competing models and baselines honestly.

Why does a baseline win MAPE? Percentage errors weight low-demand observations differently, and smoothing can help. Winning RMSE does not imply winning every objective.

Why is the validation error different from test? They contain different chronological traffic periods. A lower test error is not automatically suspicious, but leakage checks are still necessary.

Is R2 your accuracy? It is a regression score relative to an observed-mean reference, not a percentage of correct predictions.

What does the 1% threshold mean? At most one percent of validation intervals may have positive capacity deficit. It is a frequency criterion, not a deficit-volume bound or a 99% prediction interval.

Why use 20% if the target fails? It is the predeclared fallback among the tested choices, reported with its failure flag. It does not satisfy the target or prove the best possible margin.

Could you increase the margin until the target passes? Possibly, at a capacity cost. That needs a declared validation-only extended search and new evaluation. We did not silently tune beyond the original candidates using test results.

Why not allocate unlimited bandwidth and get zero shortfall? That would defeat the resource-efficiency objective. Our simple simulation already lacks a physical capacity bound; realistic cost and finite-capacity constraints are future work.

Did you reduce packet loss or latency? We did not measure those. Deficit is simulated unserved demand; packet-level behavior requires a network simulator or operational experiment.

Can it run in the cloud or an ISP? The concept can support planning, but transferring to another traffic domain requires its own data, evaluation and operational integration.

Is it real-time? The dashboard is offline replay. Fast model inference alone does not establish a real-time system: collection, preprocessing, actuation and latency budgets also matter.

What would you improve first? More institutions and seeds, rolling-origin evaluation, seasonal/statistical baselines, uncertainty/quantile forecasts, validation-calibrated capacity policies, finite capacity/cost constraints and collector-delay validation.

What did you personally contribute? Explain the decisions and code you actually understand and worked on. Be transparent about assistance if asked. Demonstrate understanding by tracing one input window through prediction, inverse scaling, metrics and allocation rather than claiming unsupported independent authorship.

20. What to revise first tonight

First learn the opening explanation and pipeline. Then work the byte-to-Mbps conversion, one window/target timeline, train-only scaling, one LSTM pass conceptually, and the allocation hand calculation. Next learn the results table, metric-dependent winners and unmet validation target. Finally rehearse the dashboard and file walkthrough aloud.

Do not spend your last preparation time memorizing every timestamp or parameter count while being unable to explain a gate, a time split or a capacity deficit. Keep this guide open as a reference; prioritize understanding the chain of reasoning.

If you cannot explain a detail, say what the implementation does, show the relevant source/configuration, and distinguish the observed result from an untested explanation. That is a stronger technical answer than inventing a certainty.

Reference material already in the project

Use docs/project_report.md for the full manuscript and exact result tables, docs/presentation/slide_notes.md for slide narration, docs/presentation/demo_script.md for rehearsal, dataset_notes.md for inspection decisions, and research_notes.txt for the chronological research record. Full-precision numbers remain in generated CSVs, not rounded presentation tables.